操作系统专题实验报告

8

2025

1 openEuler 1
1.1 1
1.2 1
1.3 2
1.4 2
1.5 4
1.6 9
1.7 .....
1.8 13
1.8.1 13
1.8.2 14
1.8.3 14
1.9 14
1.9.1 1 14
1.9.2 2 Readme..... 30
2 44
2.1 44
2.2 45
2.3 46
2.4 47
2.5 .....48.
2.6 .....50.
2.7 56
2.8 57
2.8.1 57
2.8.2 58
2.9 59
2.9.1 59
2.9.2 60
2.9.3 60
2.10 61
2.10.1 1 61
2.10.2 2 Readme..... 84
3 98
3.1 98
3.2 98
3.3 98
3.4 103
3.5 .....105.
3.6 110
3.6.1 110
3.6.2 110

3.6.3 110
3.7 111
3.7.1 1 111
3.7.2 2 Readme..... 150

1 openEuler □□□□□□

1.1

1 openEuler
2
3
4
5 PV

1.2

1 openEuler shell openEuler
2 1.1 wait
2
3 system exec PID
4 5000
system /exec PID TID,
5 5000

```
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
pid_t pid, pid1;

   /* fork a child process */
   pid = fork();

   if (pid < 0) { /* error occurred */
      fprintf(stderr, "Fork Failed");
      return 1;
   }
   else if (pid == 0) { /* child process */
      pid1 = getpid();
      printf("child: pid = %d",pid); /* A */
      printf("child: pid1 = %d",pid1); /* B */
   }
   else { /* parent process */
      pid1 = getpid();
      printf("parent: pid = %d",pid); /* C */
      printf("parent: pid1 = %d",pid1); /* D */
      wait(NULL);
   }

   return 0;
}
```

1.1

1.3

openEuler Linux openEuler Linux
openEuler shell Linux Ubuntu
openEuler shell openEuler

PID

1.4

1 , openEuler
2
2 wait() , wait() wait()
3
4 system PID PID
5 exec PID PID
6 5000
7

8
9 system TID PID,
10 exec TID PID,
11

1.5

1-1

```
[root@kp-test01 test]# gcc -o test1 test1.c
test1.c: In function ‘main’:
test1.c:25:9: warning: implicit declaration of function ‘wait’; did you mean ‘main’? [-Wimplicit-function-declaration]
         wait(NULL);
         ^~~~
         main
[root@kp-test01 test]# ./test1
parent: pid = 5089
child: pid = 0
parent: pid1 = 5088
child: pid1 = 5089
[root@kp-test01 test]# ./test1
child: pid = 0
parent: pid = 5091
child: pid1 = 5091
parent: pid1 = 5090
[root@kp-test01 test]# ./test1
parent: pid = 5093
child: pid = 0
parent: pid1 = 5092
child: pid1 = 5093
```

1-1 wait()

```
[root@kp-test01 test]# gcc -o test1 test1.c
[root@kp-test01 test]# ./test1
child: pid = 0
parent: pid = 5100
child: pid1 = 5100
parent: pid1 = 5099
[root@kp-test01 test]# ./test1
parent: pid = 5102
child: pid = 0
parent: pid1 = 5101
child: pid1 = 5102
[root@kp-test01 test]# ./test1
parent: pid = 5105
child: pid = 0
parent: pid1 = 5104
child: pid1 = 5105
```

1-1

```
[root@kp-test01 test]# gcc -o test1 test1.c
[root@kp-test01 test]# ./test1
parent: pid = 5136
child: pid = 0
parent: pid1 = 5135
child: pid1 = 5136
parent: common = 161, addr = ffffc7107004
child: common = 163, addr = ffffc7107004
[root@kp-test01 test]# ./test1
parent: pid = 5138
child: pid = 0
parent: pid1 = 5137
child: pid1 = 5138
parent: common = 161, addr = ffffc325ede4
child: common = 163, addr = ffffc325ede4
[root@kp-test01 test]# ./test1
parent: pid = 5140
child: pid = 0
parent: pid1 = 5139
child: pid1 = 5140
parent: common = 161, addr = ffffc9f3da64
child: common = 163, addr = ffffc9f3da64
```

return

```
[root@kp-test01 test]# gcc -o test1 test1.c
[root@kp-test01 test]# ./test1
parent: pid = 5210
child: pid = 0
parent: pid1 = 5209
child: pid1 = 5210
parent: common = 161, addr = ffffdaa04bf4
child: common = 163, addr = ffffdaa04bf4
common = 171, addr = ffffdaa04bf4
common = 173, addr = ffffdaa04bf4
[root@kp-test01 test]# ./test1
parent: pid = 5212
child: pid = 0
parent: pid1 = 5211
child: pid1 = 5212
parent: common = 161, addr = ffffeb7e78f4
child: common = 163, addr = ffffeb7e78f4
common = 171, addr = ffffeb7e78f4
common = 173, addr = ffffeb7e78f4
[root@kp-test01 test]# ./test1
child: pid = 0
parent: pid = 5214
child: pid1 = 5214
parent: pid1 = 5213
child: common = 163, addr = ffffeedf6c54
parent: common = 161, addr = ffffeedf6c54
common = 173, addr = ffffeedf6c54
common = 171, addr = ffffeedf6c54
```

1.9

1.9.1 □□

test1.c

```c
#include <sys/types.h>
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid, pid1;
    /* A global variable */
    int common = 162;
    /* fork a child process */
    pid = fork();

    if (pid < 0) { /* error occurred */
        fprintf(stderr, "Fork Failed");
        return 1;
    }
    else if (pid == 0) { /* child process */
        pid1 = getpid();
        printf("child: pid = %d\n", pid);   /* A */
        printf("child: pid1 = %d\n", pid1); /* B */
        printf("child: common = %d, addr = %lx\n", ++common, (unsigned long)&common); /* change common */
    }
    else { /* parent process */
        pid1 = getpid();
        printf("parent: pid = %d\n", pid);   /* C */
        printf("parent: pid1 = %d\n", pid1); /* D */
        printf("parent: common = %d, addr = %lx\n", --common, (unsigned long)&common); /* change common */
        // wait(NULL);
    }
    common += 10;
    printf("common = %d, addr = %lx\n", common, (unsigned long)&common);
    return 0;
}
```

1-1 system() exec system_call.c PID system_call system_call

```
[root@kp-test01 test]# gcc -o test1 test1.c
[root@kp-test01 test]# gcc system_call.c -o system_call
[root@kp-test01 test]# ./test1
child process1 PID: 5257
parent process PID: 5256
system_call PID: 5257
[root@kp-test01 test]# ./test1
parent process PID: 5258
child process1 PID: 5259
system_call PID: 5259
[root@kp-test01 test]#
```